

■ What is Compilation?

Compilation is the process of converting human-readable C source code into machine-executable binary code. Unlike interpreted languages (Python, JavaScript), C code must be compiled before it can run.

■ Step-by-Step Compilation Process

Stage	Tool/Phase	Input → Output	Description
1	Pre-processing (cpp)	source.c → source.i	Handles #include, #define, #ifdef macros. Expands headers and macros.
2	Compilation (cc1)	source.i → source.s	Converts C code to Assembly language. Syntax/semantic checks happen here.
3	Assembly (as)	source.s → source.o	Converts Assembly to machine-code Object file (.o / .obj).
4	Linking (ld)	source.o + libs → a.out	Links object files with libraries (stdio, math etc.) to create executable.

■ GCC Commands

```
# Compile a single file
gcc hello.c -o hello

# Run the compiled program
./hello

# Compile with all warnings enabled
gcc -Wall -Wextra hello.c -o hello

# Stop after preprocessing (view expanded code)
gcc -E hello.c -o hello.i

# Stop after compilation (view assembly)
gcc -S hello.c -o hello.s

# Stop after assembly (create object file only)
gcc -c hello.c -o hello.o

# Compile with debug info (for gdb)
gcc -g hello.c -o hello

# Compile with optimisation
gcc -O2 hello.c -o hello
```

```
# Compile multiple files
gcc main.c utils.c math.c -o program

# Link with math library
gcc program.c -o program -lm
```

■ *Note: Use `gcc -Wall` during development to catch common mistakes early. The `-o` flag specifies the output file name.*